

Collaborative Filtering: Learning the Features

In the previous "Content-Based" approach, we assumed we had the features for every movie (e.g., x_1 = Romance, x_2 = Action). But in reality, we rarely have these specific attributes for every item.

Collaborative Filtering allows us to learn these features (x) from the data itself, simultaneously with the user preferences (w and b).

1. The Intuition: Reverse Engineering

Imagine we don't know the genre of "Movie 1," but we do know the preferences of our users.

- **Alice** ($w^{(1)}$): Loves Romance, Hates Action.
- **Bob** ($w^{(2)}$): Loves Romance, Hates Action.
- **Charlie** ($w^{(3)}$): Hates Romance, Loves Action.

If Alice and Bob both rate "Movie 1" with **5 stars**, and Charlie rates it **0 stars**, the algorithm can infer that "Movie 1" must have a high "Romance" feature and a low "Action" feature.

- **Content-Based**: Given features x , learn user parameters w, b .
 - **Collaborative Filtering**: Given ratings y , learn **both** x AND w, b .
-

2. The Cost Function

To make this work, we combine the cost functions for users and movies into one massive objective.

A. Optimization for Features (x)

If we already had user parameters (w, b), we could find the best features ($x^{(i)}$) for a specific movie i by minimizing the error across all users who rated that movie:

$$J(x^{(i)}) = \frac{1}{2} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (x_k^{(i)})^2$$

B. The Combined Cost Function

To learn everything at once, we sum the error over **all valid user-movie pairs** (i, j) where a rating exists.

The total cost function $J(w, b, x)$ is:

$$J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \text{Regularization}$$

Regularization Terms:

To prevent overfitting, we add regularization for w , b , and x :

$$+\frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

3. The Algorithm: Gradient Descent

Now, x is no longer fixed input data; it is a **parameter** to be learned, just like w and b .

We minimize the cost function $J(w, b, x)$ using **Gradient Descent**.

We update all parameters iteratively:

1. **Update $w^{(j)}$** : Taking the derivative with respect to w .
2. **Update $b^{(j)}$** : Taking the derivative with respect to b .
3. **Update $x^{(i)}$** : Taking the derivative with respect to x .

By repeating these updates, the algorithm converges on a set of user preferences (w, b) and movie features (x) that accurately predict the existing ratings.

Why "Collaborative"?

This is called **Collaborative Filtering** because the users represent a "collaboration."

- When Alice and Bob rate movies, they help the system learn the feature vector x for those movies.
- Once the system learns the features x for a movie, it can use those features to predict how **Charlie** will rate it (based on Charlie's w).
- Essentially, Alice and Bob's ratings help predict Charlie's ratings, even if they have never met.